

NumPy & Pandas Cheat Sheet

NumPy & Pandas Cheat Sheet — for EDA & Exams (IT462)

Covers everything from your Assignment 2 (NumPy) & Assignment 3 (Pandas) + extra commands that are commonly asked/used in Exploratory Data Analysis (EDA). Format for every command: **What it does** → **Syntax** → **One-line use case** → **Example/Output**

```
import numpy as np
import pandas as pd
```

PART 1 — NUMPY

1. Array Creation

Command	Syntax	Use	Example → Output
Create array	<code>np.array(list)</code>	Convert Python list/nested list to ndarray	<code>np.array([10,20,30])</code> → <code>[10 20 30]</code>
Range array	<code>np.arange(start,stop,step)</code>	Sequence with fixed jump (like <code>range()</code> but ndarray)	<code>np.arange(10,30,2)</code> → <code>[10 12 14 ... 28]</code>
Equal-spaced array	<code>np.linspace(start,stop,num)</code>	<code>num</code> equally spaced points between start & stop (inclusive)	<code>np.linspace(0,1,5)</code> → <code>[0. 0.25 0.5 0.75 1.]</code>
All zeros	<code>np.zeros(shape)</code>	Placeholder/initial array of 0s	<code>np.zeros(3)</code> → <code>[0. 0. 0.]</code>
All ones	<code>np.ones(shape)</code>	Placeholder/initial array of 1s (e.g. bias init)	<code>np.ones((2,2))</code> → <code>[[1. 1.] [1. 1.]]</code>
Constant array	<code>np.full(shape, val)</code>	Array filled with a custom constant	<code>np.full((2,2), 7)</code> → <code>[[7 7] [7 7]]</code>
Identity matrix	<code>np.eye(n)</code>	Identity matrix for linear algebra	<code>np.eye(3)</code> → 3×3 identity
Random floats [0,1)	<code>np.random.rand(d0,d1)</code>	Uniform random array	<code>np.random.rand(2,2)</code>
Random ints	<code>np.random.randint(low,high,size)</code>	Random integers in range	<code>np.random.randint(1,10,5)</code>
Random normal	<code>np.random.randn(n)</code>	Standard normal (mean0, sd1) samples	Useful for simulating data
Reproducible random	<code>np.random.seed(42)</code>	Fix randomness for repeatable results	Always set before random calls
Empty array	<code>np.empty(shape)</code>	Uninitialized array (fast, garbage values)	Use only when values will be overwritten
From existing shape	<code>np.zeros_like(a)</code> / <code>np.ones_like(a)</code>	New array with same shape as <code>a</code>	Matches shape+dtype of <code>a</code>

2. Array Inspection

Command	Syntax	Use	Output meaning
Dimensions	<code>a.ndim</code>	How many axes (1D/2D/3D...)	<code>2</code> for a matrix
Shape	<code>a.shape</code>	(rows, cols) tuple	<code>(5,5)</code>
Size	<code>a.size</code>	Total number of elements	rows × cols
Data type	<code>a.dtype</code>	Type of elements stored	<code>int64</code> , <code>float64</code>
Length	<code>len(a)</code>	Number of elements along first axis	rows count
Bytes per item	<code>a.itemsize</code>	Memory (bytes) per element	e.g. <code>8</code>
Total memory	<code>a.nbytes</code>	Total memory used by array	<code>itemsize * size</code>

3. Indexing & Slicing (`A = np.arange(1,26).reshape(5,5)`)

Task	Syntax	Note
First row	<code>A[0]</code> or <code>A[:1,]</code>	Row index 0
Last row	<code>A[-1]</code> or <code>A[len(A)-1,]</code>	Negative index = from end
First column	<code>A[:,0]</code>	<code>:</code> = all rows, col 0
Last column	<code>A[:, -1]</code>	Last column
Specific element	<code>A[row,col]</code>	e.g. <code>A[2,3]</code>
Sub-matrix (slice)	<code>A[r1:r2, c1:c2]</code>	End index excluded
Middle 3×3	<code>A[1:4,1:4]</code>	Manual slicing
Reverse rows	<code>A[::-1]</code>	Flip vertically
Reverse cols	<code>A[:, ::-1]</code>	Flip horizontally
Boolean mask	<code>A[A>15]</code>	Returns flat array of matches
Even numbers	<code>A[A%2==0]</code>	Condition inside brackets
Fancy indexing	<code>A[[0,2,4]]</code>	Select specific rows by list

4. Boolean Masking & Conditional Modification

Command	Syntax	Use	Example
Condition mask	<code>a > 15</code>	Returns array of True/False	<code>[False True ...]</code>
Filter by condition	<code>a[a>15]</code>	Extract values matching condition	Only elements >15
Combine conditions (AND)	<code>a[(a>=28)&(a<=38)]</code>	Use <code>&</code> <code> </code> <code>~</code> NOT <code>and/or</code>	Must wrap each condition in <code>()</code>
Count condition matches	<code>len(a[a>35])</code> or <code>(a>35).sum()</code>	Count True values	Integer count
Conditional replace	<code>a[a>40] = 40</code>	Clip/replace values in place	Caps outliers
Clip values	<code>np.clip(a, min, max)</code>	Bound array between min & max	<code>np.clip(marks+5,0,100)</code>
Multi-condition labels	<code>np.select(conditions, choices, default)</code>	Assign categories/grades based on multiple rules	Used for grade assignment
Elementwise if-else	<code>np.where(cond, val_if_true, val_if_false)</code>	Vectorized ternary	<code>np.where(a>50,"Pass","Fail")</code>

5. Data Type Conversion

Command	Syntax	Use
Convert dtype	<code>a.astype('int32')</code>	Change storage type (memory/precision tradeoff)
Check type	<code>a.dtype</code>	Verify before/after conversion
Why convert?	—	Saves memory, avoids overflow, needed before math ops on strings

6. Reshaping & Combining

Command	Syntax	Use
Reshape	<code>a.reshape(rows,cols)</code>	Change shape, same total elements
Flatten to 1D	<code>a.flatten()</code> / <code>a.ravel()</code>	<code>flatten</code> = copy, <code>ravel</code> = view (faster)
Transpose	<code>a.T</code> or <code>np.transpose(a)</code>	Swap rows ↔ columns
Stack vertically	<code>np.vstack([a,b])</code>	Combine along rows
Stack horizontally	<code>np.hstack([a,b])</code>	Combine along columns
Concatenate	<code>np.concatenate([a,b], axis=0)</code>	General join along an axis
Split array	<code>np.split(a, n)</code>	Break into n equal parts
Auto-dimension	<code>a.reshape(-1,1)</code>	<code>-1</code> = “figure this dimension out” (common for ML column vectors)

7. Statistics & Aggregation

Command	Syntax	Use
Mean	<code>np.mean(a)</code> / <code>a.mean()</code>	Average
Median	<code>np.median(a)</code>	Middle value
Variance	<code>np.var(a)</code>	Spread of data
Std deviation	<code>np.std(a)</code>	Spread (sqrt of variance)
Min / Max	<code>np.min(a)</code> / <code>np.max(a)</code>	Extremes
Range	<code>a.max()-a.min()</code>	Spread indicator
Sum	<code>np.sum(a)</code>	Total
Average (weighted)	<code>np.average(a, weights=w)</code>	Weighted mean
Percentile	<code>np.percentile(a, [25,50,75])</code>	Quartiles
Cumulative sum	<code>np.cumsum(a)</code>	Running total
Cumulative product	<code>np.cumprod(a)</code>	Running product
Difference between elements	<code>np.diff(a)</code>	Rate of change
Correlation coefficient	<code>np.corrcoef(a,b)</code>	Linear relationship between arrays
Row/col-wise stats	<code>a.sum(axis=0)</code> / <code>a.sum(axis=1)</code>	<code>axis=0</code> → down columns, <code>axis=1</code> → across rows

8. Broadcasting & Arithmetic

Concept	Rule	Example
Broadcasting	Smaller array is “stretched” to match bigger one if trailing dims match or are 1	<code>(3,3) + (3,)</code> works
Element-wise ops	<code>+</code> <code>-</code> <code>*</code> <code>/</code> work directly on arrays	<code>a+b</code> , <code>a*b</code>
Matrix multiplication	<code>a @ b</code> or <code>np.dot(a,b)</code>	True matrix product (not element-wise)
Broadcasting fails when	Shapes incompatible (e.g. <code>(3,)</code> vs <code>(2,)</code>)	Raises <code>ValueError</code>

9. Sorting, Searching & Unique Values

Command	Syntax	Use
Sort array	<code>np.sort(a)</code>	Ascending sort (returns copy)

Sort indices	<code>np.argsort(a)</code>	Indices that would sort the array
Index of max/min	<code>np.argmax(a) / np.argmin(a)</code>	Position of largest/smallest value
Unique values	<code>np.unique(a)</code>	Distinct values, sorted
Unique + counts	<code>np.unique(a, return_counts=True)</code>	Value + frequency (like <code>value_counts</code>)
Check any/all	<code>np.any(cond) / np.all(cond)</code>	At least one / every element satisfies condition
Check NaN	<code>np.isnan(a)</code>	Detect missing/invalid values
Membership test	<code>np.isin(a, [list])</code>	Check if elements exist in another array

PART 2 — PANDAS

1. Reading, Writing & Creating Data

Command	Syntax	Use
Create DataFrame	<code>pd.DataFrame(dict)</code>	Build DF from dict of lists
Create Series	<code>pd.Series(list)</code>	Build single column
Read CSV	<code>pd.read_csv('file.csv')</code>	Load CSV file
Read CSV with index	<code>pd.read_csv('f.csv', index_col=0)</code>	Use first column as row labels
Write CSV	<code>df.to_csv('file.csv', index=False)</code>	Save without extra index column
Read Excel	<code>pd.read_excel('file.xlsx', sheet_name='Sheet1')</code>	Load Excel sheet
Write Excel	<code>df.to_excel('file.xlsx', index=False)</code>	Save to Excel
Read JSON	<code>pd.read_json('file.json')</code>	Load JSON data

2. DataFrame Inspection

Command	Syntax	Use
View full data	<code>df</code> or <code>print(df)</code>	Display DataFrame
First n rows	<code>df.head(n)</code>	Preview start (default 5)
Last n rows	<code>df.tail(n)</code>	Preview end
Random sample	<code>df.sample(n)</code>	Random n rows (good for EDA sanity check)
Shape	<code>df.shape</code>	(rows, columns)
Column names	<code>df.columns</code>	List of column labels
Data types	<code>df.dtypes</code>	Type per column
Summary info	<code>df.info()</code>	dtypes + non-null counts + memory
Statistical summary	<code>df.describe()</code>	count, mean, std, min, quartiles, max (numeric cols)
Describe all (incl. object)	<code>df.describe(include='all')</code>	Stats for numeric + categorical
Memory usage	<code>df.memory_usage(deep=True)</code>	Bytes used per column
Index	<code>df.index</code>	Row label range/values
Unique count per column	<code>df.nunique()</code>	Distinct value count

3. Selection & Indexing

Task	Syntax	Note
Select 1 column	<code>df['A']</code>	Returns Series
Select multiple columns	<code>df[['A', 'B']]</code>	Returns DataFrame (note double brackets)
Select row by label	<code>df.loc['r3']</code>	Label-based, inclusive on slices
Select row by position	<code>df.iloc[2]</code>	Position-based, exclusive end on slices
Single value	<code>df.loc['r4', 'B']</code>	Row label + column label
Slice rows+cols (label)	<code>df.loc['r2':'r4', 'A':'B']</code>	Both ends included
Slice rows+cols (position)	<code>df.iloc[1:4, 0:2]</code>	End excluded (like Python lists)
Boolean filter	<code>df[df['A']>25]</code>	Rows matching condition
Multiple conditions	<code>df[(df['A']>25) & (df['B']<50)]</code>	Use <code>&</code> , <code> </code> , <code>~</code> with parentheses
isin filter	<code>df[df['C'].isin(['x', 'z'])]</code>	Match against a list of values
between filter	<code>df[df['B'].between(20, 45)]</code>	Inclusive range filter
query syntax	<code>df.query('A > 25 and C=="x"')</code>	Alternative readable filtering
Drop column	<code>df.drop('col', axis=1)</code>	Remove column
Drop row	<code>df.drop(index)</code>	Remove row by label
Set index	<code>df.set_index('col')</code>	Make a column the row index
Reset index	<code>df.reset_index(drop=True)</code>	Back to default 0,1,2... index, drop old one

4. Handling Missing Data

Command	Syntax	Use
Detect missing (mask)	<code>df.isnull()</code> / <code>df.isna()</code>	True where NaN
Detect non-missing	<code>df.notnull()</code>	True where value exists
Count missing per column	<code>df.isnull().sum()</code>	Column-wise NaN count
Total missing count	<code>df.isnull().sum().sum()</code>	Whole DataFrame NaN count
Drop rows with any NaN	<code>df.dropna()</code>	Removes any row with ≥ 1 missing value
Drop rows fully empty	<code>df.dropna(how='all')</code>	Removes only fully-NaN rows
Drop by column	<code>df.dropna(subset=['col'])</code>	Drop rows missing in specific column(s)
Fill with constant	<code>df.fillna(0)</code>	Replace NaN with fixed value
Fill with mean	<code>df['A'].fillna(df['A'].mean())</code>	Common for numeric columns
Fill with median	<code>df['A'].fillna(df['A'].median())</code>	Robust to outliers
Fill with mode	<code>df['A'].fillna(df['A'].mode()[0])</code>	Common for categorical columns
Forward/backward fill	<code>df.fillna(method='ffill')</code> / <code>'bfill'</code>	Carry previous/next value forward
Interpolate	<code>df.interpolate()</code>	Estimate missing values linearly
dropna vs fillna	—	Use <code>dropna()</code> when missing data is small/random and won't bias results; use <code>fillna()</code> when many rows are missing and you can't afford to lose data — fill with mean/median/mode instead.

5. Duplicates & Data Cleaning

Command	Syntax	Use
Find duplicate rows	<code>df.duplicated()</code>	True for repeated rows (keeps first as False)
Count duplicates	<code>df.duplicated().sum()</code>	Total duplicate row count
Remove duplicates	<code>df.drop_duplicates()</code>	Keeps first occurrence, drops rest
Remove dup by subset	<code>df.drop_duplicates(subset=['col'])</code>	Based on specific column(s)
Type conversion	<code>df['col'].astype(float)</code>	String $\rightarrow$ numeric etc.
Convert to datetime	<code>pd.to_datetime(df['col'])</code>	Parse date strings into datetime objects
Convert to boolean/map	<code>df['col'].map({'Yes':True, 'No':False})</code>	Label $\rightarrow$ boolean/numeric
Rename columns	<code>df.rename(columns={'old':'new'})</code>	Relabel columns
Replace values	<code>df['col'].replace('old', 'new')</code>	Substitute specific values
Why type conversion matters	—	Stats (mean/sum/comparisons) only work correctly on the right dtype; text can't be used in math, unparsed dates can't be sorted/compared.

6. Sorting & Ranking

Command	Syntax	Use
Sort ascending	<code>df.sort_values('col')</code>	Default ascending
Sort descending	<code>df.sort_values('col', ascending=False)</code>	Highest first
Sort by multiple cols	<code>df.sort_values(['A', 'B'], ascending=[True, False])</code>	Tie-breaker sorting
Sort by index	<code>df.sort_index()</code>	Order rows by index label
Rank values	<code>df['col'].rank(method='min', ascending=False)</code>	1 = highest; <code>method</code> controls tie handling (<code>min</code> , <code>dense</code> , <code>average</code> , <code>first</code>)
Top-N rows	<code>df.nlargest(n, 'col')</code>	Fastest way to get top N
Bottom-N rows	<code>df.nsmallest(n, 'col')</code>	Fastest way to get bottom N
Reset row numbers	<code>df.reset_index(drop=True)</code>	Clean sequential index after sorting/filtering

7. GroupBy & Aggregation

Command	Syntax	Use
Group + single stat	<code>df.groupby('Dept')['Salary'].mean()</code>	Mean per group
Group + sum	<code>df.groupby('Dept')['Salary'].sum()</code>	Total per group
Group size (row count)	<code>df.groupby('Dept').size()</code>	Count of rows per group (incl. NaN)
Group count (non-null)	<code>df.groupby('Dept')['col'].count()</code>	Non-null count per group
Multiple stats (list)	<code>df.groupby('Dept')['Exp'].agg(['min', 'max', 'mean'])</code>	Several aggregations on one column
Multiple stats (dict)	<code>df.groupby('Dept').agg({'Salary':'mean', 'Exp':'sum'})</code>	Different aggregation per column
transform (row-preserving)	<code>df.groupby('Dept')['Salary'].transform('mean')</code>	Broadcasts group result back to every row (same row count)
Filter groups	<code>df.groupby('Dept').filter(lambda x: x['Salary'].mean()>55000)</code>	Keep only groups meeting condition
agg vs transform	—	<code>agg()</code> collapses each group into one summary row (fewer rows out). <code>transform()</code> keeps original row count, attaching the group result to every row — useful for comparing an individual to its group average.

8. Descriptive Statistics & Value Counts

Command	Syntax	Use
Mean/Median/Std	<code>df['col'].mean() / .median() / .std()</code>	Central tendency & spread
Min/Max	<code>df['col'].min() / .max()</code>	Extremes
Quantile	<code>df['col'].quantile([0.25, 0.5, 0.75])</code>	Percentiles
Correlation matrix	<code>df.corr(numeric_only=True)</code>	Pairwise linear relationship (-1 to 1) between numeric cols
Covariance matrix	<code>df.cov(numeric_only=True)</code>	How two variables vary together (unscaled)
Value frequency	<code>df['col'].value_counts()</code>	Count of each unique value, sorted desc
Value % share	<code>df['col'].value_counts(normalize=True)</code>	Proportion instead of raw count
Unique count per col	<code>df.nunique()</code>	Distinct values per column
Unique values list	<code>df['col'].unique()</code>	Array of distinct values
Range	<code>df['col'].max() - df['col'].min()</code>	Spread
Skewness	<code>df['col'].skew()</code>	Asymmetry of distribution
Kurtosis	<code>df['col'].kurt()</code>	"Tailedness" of distribution

9. Transformation: Apply, Map, Replace, Cut

Command	Syntax	Use
apply on column	<code>df['col'].apply(func)</code>	Run function on every value in a Series
apply with lambda	<code>df['col'].apply(lambda x: 'High' if x>=85 else 'Low')</code>	Custom conditional logic
apply on rows	<code>df.apply(func, axis=1)</code>	Row-wise function (access multiple columns)
map (Series only)	<code>df['col'].map(dict_or_func)</code>	Value replacement via dict or function, Series-only
map on whole DataFrame	<code>df.map(func)</code>	Apply function to every single cell (pandas ≥2.1; was <code>applymap</code>)
replace values	<code>df['col'].replace('old', 'new')</code>	Direct substitution of specific value(s)
Bin into ranges	<code>pd.cut(df['col'], bins=[0,50,100], labels=['Low', 'High'])</code>	Convert continuous → categorical bins (equal-width)
Bin into quantiles	<code>pd.qcut(df['col'], q=4)</code>	Bins with equal number of observations
apply vs map vs (df-level) map	—	<code>apply()</code> → function on a whole column/row (calculations, custom conditions). <code>map()</code> → Series-only value replacement (often via dict). DataFrame-level <code>map()</code> (old <code>applymap</code>) → apply function to every individual cell of the whole DataFrame.

10. String & Datetime Accessors (very common in EDA)

Command	Syntax	Use
Lowercase/uppercase	<code>df['col'].str.lower() / .str.upper()</code>	Text normalization
Strip whitespace	<code>df['col'].str.strip()</code>	Clean leading/trailing spaces
Contains substring	<code>df['col'].str.contains('abc')</code>	Boolean filter for text search
Split string	<code>df['col'].str.split(',')</code>	Break string into list
Replace in string	<code>df['col'].str.replace('a', 'b')</code>	Substring substitution
Extract year	<code>df['date'].dt.year</code>	After <code>pd.to_datetime()</code> , get year
Extract month/day	<code>df['date'].dt.month / .dt.day</code>	Date component extraction
Day of week	<code>df['date'].dt.day_name()</code>	e.g. "Monday"
Date difference	<code>df['end'] - df['start']</code>	Returns Timedelta

11. Merging, Joining & Concatenating

Command	Syntax	Use
Inner join	<code>pd.merge(df1, df2, on='ID', how='inner')</code>	Keep only IDs present in both — clean, complete records
Left join	<code>pd.merge(df1, df2, on='ID', how='left')</code>	Keep all of df1; NaN where df2 has no match
Right join	<code>pd.merge(df1, df2, on='ID', how='right')</code>	Keep all of df2; NaN where df1 has no match
Outer join	<code>pd.merge(df1, df2, on='ID', how='outer')</code>	Keep all IDs from both; NaN wherever missing — good for spotting gaps
Concat rows	<code>pd.concat([df1, df2], axis=0)</code>	Stack DataFrames vertically
Concat columns	<code>pd.concat([df1, df2], axis=1)</code>	Join side-by-side by position (not by key!)
Join on index	<code>df1.join(df2)</code>	Merge using index instead of a column
merge vs concat	—	<code>merge()</code> combines using a common key column (like SQL join). <code>concat()</code> just stacks data together (rows or columns) without matching on any key.
inner vs outer in EDA	—	Inner join → only complete, matched records (cleaner). Outer join → everything from both sources, revealing missing/unmatched data (better for spotting gaps).

12. Pivot Tables & Cross-tabulation

Command	Syntax	Use
Pivot table (1 agg)	<code>pd.pivot_table(df, values='Sales', index='Region', columns='Product', aggfunc='mean')</code>	Summarize numeric column across two categorical dimensions
Pivot table (multi-col agg)	<code>pd.pivot_table(df, index='Region', values=['Sales', 'Profit'], aggfunc={'Sales': 'mean', 'Profit': 'sum'})</code>	Different aggregation per numeric column
Simple reshape pivot	<code>df.pivot(index='A', columns='B', values='C')</code>	No aggregation, must have unique index/column combos
Cross tabulation	<code>pd.crosstab(df['Region'], df['Product'])</code>	Frequency count of category combinations
Crosstab proportions	<code>pd.crosstab(df['Region'], df['Product'], normalize=True)</code>	Percentages instead of counts
Crosstab with totals	<code>pd.crosstab(df['Region'], df['Product'], margins=True)</code>	Adds row/column/grand totals
Melt (wide→long)	<code>pd.melt(df, id_vars='ID', value_vars=['A', 'B'])</code>	Unpivot columns into rows
Stack / Unstack	<code>df.stack()</code> / <code>df.unstack()</code>	Pivot column levels ↔ row index levels
pivot_table vs groupby	—	<code>pivot_table()</code> is preferred when you want a matrix-style table (rows × columns) comparing two categorical variables at once — easier visual comparison than a groupby's flat output.
pivot_table vs crosstab	—	<code>pivot_table()</code> aggregates a numeric column with any function (mean, sum...) and gives full control. <code>crosstab()</code> is mainly for counting frequency of category combinations (default = count).

PART 3 — EXTRA EDA-ESSENTIAL COMMANDS (not in assignments but exam-important)

Command	Syntax	Use
Correlation heatmap prep	<code>df.corr()</code>	Precede with <code>sns.heatmap(df.corr(), annot=True)</code> for visual EDA
Outlier detection (IQR)	<code>Q1=df['c'].quantile(.25); Q3=df['c'].quantile(.75); IQR=Q3-Q1</code>	Standard EDA outlier bound: <code>Q1-1.5*IQR</code> , <code>Q3+1.5*IQR</code>
Z-score outliers	<code>(df['c']-df['c'].mean())/df['c'].std()</code>	Values with
Histogram	<code>df['col'].plot(kind='hist')</code> or <code>df.hist()</code>	Visualize distribution shape
Boxplot	<code>df.boxplot(column='col')</code>	Visualize spread & outliers
Scatter plot	<code>df.plot.scatter(x='A', y='B')</code>	Visualize relationship between two numeric vars
Correlation with target	<code>df.corr()['target'].sort_values()</code>	Rank features by relation to target variable
Cross-column apply	<code>df.apply(lambda row: row['A']+row['B'], axis=1)</code>	Row-wise custom calculation
One-hot encoding	<code>pd.get_dummies(df['col'])</code>	Convert categorical column into binary columns (for ML)
Label encoding via map	<code>df['col'].map({'Low':0, 'Med':1, 'High':2})</code>	Ordinal categorical → numeric
Normalize (min-max)	<code>(df['c']-df['c'].min())/(df['c'].max()-df['c'].min())</code>	Scale values to [0,1]
Standardize (z-score)	<code>(df['c']-df['c'].mean())/df['c'].std()</code>	Scale to mean 0, std 1
Check column value range quickly	<code>df.describe().T</code>	Transposed summary, easier to read many columns
Filter top-correlated features	<code>df.corr().abs().unstack().sort_values(ascending=False)</code>	Find strongest relationships in a dataset
Column-wise apply of NumPy func	<code>df.apply(np.mean)</code>	Combine NumPy + pandas
Convert DataFrame to NumPy	<code>df.to_numpy()</code> or <code>df.values</code>	For feeding into NumPy/ML functions
Random train/test style split	<code>df.sample(frac=0.8) then df.drop(train.index)</code>	Quick manual split without sklearn
Chained filtering with query	<code>df.query('Sales > 100 and Region == "East"')</code>	Cleaner alternative to bracket-boolean chains
Cross-check shape after ops	<code>df.shape</code> (before) vs <code>df.shape</code> (after)	Sanity check row/col loss during cleaning
Column selection by dtype	<code>df.select_dtypes(include='number')</code>	Grab only numeric (or <code>exclude=</code>) columns — handy before <code>.corr()</code>
Apply function across axis with agg names	<code>df.agg(['mean', 'std', 'min', 'max'])</code>	Multiple stats on the whole DataFrame at once

🔑 Quick Recall Table — Most Exam-Asked One-Liners

Concept	One-liner
Shape of data	<code>df.shape</code>
Nulls per column	<code>df.isnull().sum()</code>
Fill numeric NaN	<code>df['c'].fillna(df['c'].mean())</code>
Drop duplicate rows	<code>df.drop_duplicates()</code>
Convert to datetime	<code>pd.to_datetime(df['c'])</code>
Sort descending	<code>df.sort_values('c', ascending=False)</code>
Group mean	<code>df.groupby('g')['c'].mean()</code>

Row-preserving group stat	<code>df.groupby('g')['c'].transform('mean')</code>
Frequency of category	<code>df['c'].value_counts()</code>
Correlation matrix	<code>df.corr(numeric_only=True)</code>
Merge on key (all matched)	<code>pd.merge(df1,df2,on='ID',how='inner')</code>
Merge keep everything	<code>pd.merge(df1,df2,on='ID',how='outer')</code>
Stack rows	<code>pd.concat([df1,df2],axis=0)</code>
Region×Product avg table	<code>pd.pivot_table(df,values='S',index='R',columns='P',aggfunc='mean')</code>
Category frequency table	<code>pd.crosstab(df['R'],df['P'])</code>
NumPy conditional label	<code>np.select(conditions, choices, default)</code>
NumPy filter	<code>a[a>15]</code>
NumPy clip outliers	<code>np.clip(a, low, high)</code>
NumPy stats bundle	<code>a.mean(), a.std(), a.min(), a.max()</code>

🔗 Exam Tips

- `loc` = **label**-based & end-**inclusive**; `iloc` = **position**-based & end-**exclusive**. This is the #1 confusion point.
- `agg()` shrinks rows (summary); `transform()` keeps all rows (broadcasts group result back).
- `merge()` = key-based join (SQL-style); `concat()` = simple stack, no key matching.
- Always wrap multiple boolean conditions in `()` and use `&` | `~`, never `and/or`.
- `dropna()` → few missing values, safe to delete. `fillna()` → many missing, need to preserve rows.
- Use `numeric_only=True` in `.corr()` / `.cov()` on pandas ≥2.0 to avoid errors with text columns.